

BABEŞ–BOLYAI TUDOMÁNYEGYETEM KOLOZSVÁR
MATEMATIKA ÉS INFORMATIKA KAR
INFORMATIKA SZAK

Projektjelentés

Intelligens Otthoni Energiarendszerek Optimalizálása Természetből Inspirált Metaheurisztikák Alkalmazásával



TÉMAVEZETŐ:
GASKÓ NOÉMI

SZERZŐ:
ERDEI KRISTÓF-ZOLTÁN

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Project Report

Optimization of Smart Home Energy Systems with Nature-Inspired Metaheuristics



ADVISOR:
NOÉMI GASKÓ

AUTHOR:
KRISTÓF-ZOLTÁN ERDEI

2026

UNIVERSITATEA BABEȘ–BOLYAI, CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

Raport de proiect

Optimizarea Sistemelor de Energie pentru Locuințe prin Metaheuristici Inspirare din Natură



CONDUCĂTOR ȘTIINȚIFIC:
NOÉMI GASKÓ

ABSOLVENT:
KRISTÓF-ZOLTÁN ERDEI

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Project Report

Optimization of Smart Home Energy Systems with Nature-Inspired Metaheuristics

Abstract

This project report studies the optimization of a Home Energy Management System (HEMS) using nature-inspired metaheuristics. We formulate a cost-minimization scheduling problem with practical constraints (e.g., appliance timing, PV generation uncertainty, and battery operation) and compare four approaches: Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Grey Wolf Optimizer (GWO), and a hybrid GA–PSO method. Experiments are evaluated on multiple seasonal scenarios (winter, summer, and cloudy days) using repeated runs to assess robustness. The results highlight that algorithm choice depends on the operating scenario: exploration-focused methods perform better under uncertainty, while constraint-heavy cases benefit from stronger exploitation. Finally, we summarize implementation recommendations and the expected economic and environmental impact.

2026

KRISTÓF-ZOLTÁN ERDEI

ADVISOR:
NOÉMI GASKÓ

Tartalomjegyzék

1. Bevezetés és Problémafelvetés	2
1.1. A projekt célja	2
1.2. A keresletoldali szabályozás (DSM) jelentősége	2
1.3. Problémafelvetés és motiváció	3
1.4. Kutatási célok és hozzájárulások	3
2. Irodalmi Áttekintés és Fogalmi Alapok	4
2.1. Irodalmi háttér	4
2.2. Definíció: Peak-to-Average Ratio	4
3. Alkalmazott Algoritmusok	6
3.1. Genetikus Algoritmus (GA)	6
3.2. Particle Swarm Optimization (PSO)	6
3.3. Grey Wolf Optimizer (GWO)	7
3.4. Hibrid GA-PSO	7
4. Kísérleti Eredmények	8
4.1. Kísérleti scénáriók	8
4.2. Téli scénárió eredményei	8
4.3. Nyári scénárió eredményei	9
4.4. Viharos/Borús scénárió eredményei	9
4.5. Eredmények elemzése	10
4.6. Implementációs részletek	10
4.7. Ábrák	10
5. Következtetések és Ajánlások	11
5.1. Egy nap részletes esetelemzése	11
5.2. Főbb megállapítások	12
5.3. Gazdasági megtérülés	13
5.4. Környezeti hatás	13
5.5. Jövőbeli kutatási irányok	13
5.5.1. Energiaközösségek (REC)	13
5.5.2. Multi-objective optimalizálás	13
5.5.3. Gépi tanulás integráció	14
5.5.4. V2G integráció	14

TARTALOMJEGYZÉK

5.6. Implementációs ajánlások	14
5.6.1. Kis háztartásoknak (< 3 fő)	14
5.6.2. Közepes háztartásoknak (3–5 fő)	14
A. Fontosabb programkódok listája	15
A.1. HEMS kódbázis	15
A.2. Mintakódrészlet	15

1. fejezet

Bevezetés és Problémafelvetés

***Összefoglaló:** A fejezet bemutatja a HEMS célját és a keresletoldali szabályozás jelentőségét.*

A globális energetikai átalakulás, a decentralizált termelés térnyerése és a fenntarthatósági elvárások a Smart Grid technológiák felé terelik a villamosenergia-hálózatokat. A HEMS kulcsszereplő a lakossági oldalon: kétirányú kommunikációt és energiaáramlást kezel, összehangolja a fogyasztókat, tárolást és megújuló termelést.

1.1. A projekt célja

A projekt elsődleges célja egy olyan HEMS optimalizálási modell kidolgozása, amely képes a háztartási eszközök összehangolt vezérlésére és a megújuló források optimális kihasználására. A rendszer emellett biztosítja az energiatárolók intelligens irányítását, miközben minimalizálja a villamosenergia-költséget. Fontos szempont továbbá a hálózati terhelési csúcsok csökkentése (PAR redukció) a felhasználói komfort biztosítása mellett.

1.2. A keresletoldali szabályozás (DSM) jelentősége

A terhelésselátolás (Load Shifting) számos előnnyel jár [Fadaee és Radzi, 2012]. A legfontosabb a közvetlen, akár 20–40%-os költségmegtakarítás, de emellett hozzájárul a hálózati csúcsok csökkentéséhez is. Szélesebb körben alkalmazva mérsékli a fosszilis erőművek igénybevételét, ezáltal pedig csökkenti a károsanyag-kibocsátást.

1.3. Problémafelvetés és motiváció

A HEMS optimalizálás a gyakorlatban több, egymással versengő cél kezelését jelenti: a költség minimalizálását, a felhasználói komfort fenntartását és a hálózati terhelés csökkentését. A lakossági szektorban a fogyasztás időben erősen ingadozó, míg a napelemes termelés időjárásfüggő és bizonytalan. Ezért a feladat nem egyszerű ütemezési probléma, hanem korlátokkal terhelt, vegyes (diszkrét és folytonos) döntési változókat tartalmazó optimalizációs modellként kezelhető [Fadaee és Radzi, 2012; Ismail et al., 2014]. A téma aktualitását jelzi a megújuló kapacitások növekedése és a dinamikus tarifák terjedése, amelyek a fogyasztói oldalon rugalmas szabályozást tesznek lehetővé. További háttérként lásd az [európai energiapolitikai összefoglalót](#).

1.4. Kutatási célok és hozzájárulások

A dolgozat célja egy olyan optimalizációs keret bemutatása, amely természetből inspirált metaheurisztikák (GA, PSO, GWO, hibrid GA–PSO) segítségével csökkenti a napi villamosenergia-költséget, miközben betartja a fizikai és hálózati korlátokat. A megközelítés a szakirodalmi irányokhoz illeszkedik, de a különböző scénáriókban végzett összehasonlítás révén gyakorlati szempontokat is hangsúlyoz [Lazzari et al., 2023].

2. fejezet

Irodalmi Áttekintés és Fogalmi Alapok

Összefoglaló: Áttekintjük a HEMS terület fő irányait, valamint bevezetünk néhány alapfogalmat a modell értelmezéséhez.

2.1. Irodalmi háttér

A HEMS rendszerek optimalizálása tipikusan korlátozott, nemlineáris feladat, amelyben a helyi optimumok és a bizonytalan bemenetek miatt a metaheurisztikus megoldások széles körben elterjedtek [Ismail et al., 2014]. A sztochasztikus termelés és a tárolás kezelési nehézségei a rendszer rugalmasságát és korlátkezelést helyezik előtérbe [Fadaee és Radzi, 2012]. A gyakorlatban ez azt jelenti, hogy a különböző algoritmusok trade-offokkal dolgoznak: gyorsaság vs. pontosság, illetve átlagos vs. legjobb megoldás.

2.2. Definíció: Peak-to-Average Ratio

2.1. Értelmezés (Peak-to-Average Ratio – PAR). A PAR érték a csúcsfogyasztás és az átlagfogyasztás hanyadosa, amely a hálózati terheles ingadozást jelzi:

$$PAR = \frac{\max_{t \in T} P_{grid}(t)}{\frac{1}{T} \sum_{t=1}^T P_{grid}(t)} \quad (2.1)$$

Minél alacsonyabb a PAR, annal kiegyensúlyozottabb a fogyasztási profil.

2.2. Tétel. Ha $P_{grid}(t) \geq 0$ minden $t \in T$ időpontban, akkor $PAR \geq 1$.

Bizonyítás. A definíció szerint az átlagfogyasztás nem lehet nagyobb, mint a csúcsfogyasztás.

Ezért $\frac{1}{T} \sum_{t=1}^T P_{grid}(t) \leq \max_{t \in T} P_{grid}(t)$, ami alapján a hanyados legalább 1. ■

2. FEJEZET: IRODALMI ATTEKINTES ES FOGALMI ALAPOK

2.3. Példa. Ha egy adott napon a csucsfogyasztás 4 kW, az átlag pedig 2 kW, akkor $PAR = 2$. Ez egyenetlen terhelést jelez, amelyet terheles-eltoással csökkenteni lehet.

2.4. *Megjegyzés.* A PAR csökkentése nem csak költségcsökkentéssel jár, hanem a hálózati csúcsok merseklesevel is, ami a szolgáltató számára is előnyös.

3. fejezet

Alkalmazott Algoritmusok

Összefoglaló: Négy metaheurisztikát és egy hibrid eljárást alkalmaztunk: GA, PSO, GWO és GA-PSO hibrid.

3.1. Genetikus Algoritmus (GA)

A genetikus algoritmusok alapötletét Holland klasszikus munkája alapozta meg [Holland, 1975]. A módszer fő előnye, hogy jól kezeli a diszkrét változókat és robusztus megoldást nyújt [Holland, 1975], bár hátránya a lassabb konvergencia. A mérések alapján az algoritmus átlagosan 15–20%-os költségcsökkenést eredményez.

Az implementált algoritmus működése a következő lépésekből áll. Elsőként generálunk N darab véletlen egyedet, amelyek a kezdeti populációt alkotják, majd minden egyedhez kiszámítjuk a költségfüggvény alapján a fitness értéket. Ezt követi a fő ciklus, amely G generáción keresztül fut. Minden iterációban az elitizmus elve alapján a legjobb két egyed automatikusan továbbjut. A szülőket tournament módszerrel választjuk ki, majd aritmetikai átlagolással és zaj hozzáadásával keresztezzük őket. A sokszínűség fenntartása érdekében Gauss-eloszlású véletlen módosítással mutációt hajtunk végre. Végül az új populáció létrehozása és a határok ellenőrzése után az algoritmus visszatér a legjobb megoldással.

3.2. Particle Swarm Optimization (PSO)

A PSO eredeti formáját Kennedy és Eberhart vezette be [Kennedy és Eberhart, 1995]. Sebesség-frissítési egyenlet:

$$V_i^{k+1} = w \cdot V_i^k + c_1 r_1 (P_{best,i} - X_i^k) + c_2 r_2 (G_{best} - X_i^k) \quad (3.1)$$

3. FEJEZET: ALKALMAZOTT ALGORITMUSOK

Pozíció-frissítés:

$$X_i^{k+1} = X_i^k + V_i^{k+1} \quad (3.2)$$

Az (3.1) és (3.2) egyenletek adják a PSO alapidinamikáját. Előny: gyors, egyszerű. Hátrány: lokális optimumra hajlamos. Teljesítmény: $\approx 15\text{--}18\%$ költségcsökkenés.

3.3. Grey Wolf Optimizer (GWO)

A GWO algoritmust Mirjalili és szerzőtársai javasolták [Mirjalili et al., 2014].

Hierarchikus struktúra: $\alpha, \beta, \delta, \omega$. Pozíció-frissítés:

$$\vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}_\alpha - \vec{X}| \quad (3.3)$$

$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha \quad (3.4)$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \quad (3.5)$$

Előny: kevesebb paraméter, jobb exploration-exploitation egyensúly. Teljesítmény: $\approx 18\text{--}20\%$ költségcsökkenés.

3.4. Hibrid GA-PSO

A GA és PSO kombinálása ismert hibridizációs irány [Kumar et al., 2015].

A hibridizáció logikája a két módszer erősségeit ötvözi: a keresés első fázisában a PSO végzi a gyors feltérképezést a sebesség-frissítési egyenletek segítségével. Ezt követően a GA veszi át az irányítást, amely keresztezéssel és mutációval biztosítja a megoldások diverzifikációját. A két fázis közötti váltás adaptív módon történik, így az algoritmus képes elkerülni a korai konvergenciát, elérve ezzel mintegy $18\text{--}22\%$ -os költségcsökkenést.

4. fejezet

Kísérleti Eredmények

***Összefoglaló:** Három scenárióban (tél, nyár, borús) 20-20 futással értékeltük az algoritmusokat; a nyers eredmények CSV/JSON formában mentve vannak.*

4.1. Kísérleti scenáriók

A kísérleteket három, markánsan eltérő scenárióban futtattuk le (20-20 alkalommal), hogy a rendszer robusztusságát vizsgáljuk. A **téli** esetet az alacsony PV termelés és a magasabb alapfogyasztás jellemzi, ami nagy hálózati függést eredményez. A **nyári** időszakban ezzel szemben magas, csúcsban akár 6 kW-os PV termeléssel és alacsonyabb fogyasztással számolhatunk, ami jelentős visszatáplálási potenciált rejt. Harmadikként egy **viharos/borús** (cloudy) napot szimuláltunk, ahol az ingadozó PV termelés és a magas zaj miatt az akkumulátor dinamikus kezelése válik szükségessé, ezáltal stressztesztnek vetve alá az algoritmusokat.

A reprodukálhatóság érdekében minden futás determinisztikus bázis-seeddel (42) indul, amelyet a 20 ismétléshez futásonként eltoltunk. A nyers eredményeket és a futási idők listáit CSV/JSON formában mentettük a `results_winter`, `results_summer` és `results_cloudy` fájlokba.

4.2. Téli scenárió eredményei

Az alapvonal (akku és időzítés nélkül) télen átlagosan **42.09 RON/nap**.

4. FEJEZET: KÍSÉRLETI EREDMÉNYEK

4.1. táblázat. Téli Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	37.05	34.92	0.31
GA (P=100)	37.25	35.06	0.62
PSO	36.87	34.56	0.21
GWO	35.40	33.91	0.44
Hybrid	36.28	34.42	0.24

4.3. Nyári Szenárió eredményei

Az alapvonal (akku és időzítés nélkül) nyáron átlagosan **12.48 RON/nap**.

4.2. táblázat. Nyári Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	4.08	3.74	0.31
GA (P=100)	3.93	3.64	0.62
PSO	6.67	5.20	0.20
GWO	5.17	4.21	0.43
Hybrid	5.88	4.93	0.25

4.4. Viharos/Borús Szenárió eredményei

4.3. táblázat. Viharos/Borús Nap Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	14.13	13.58	0.31
GA (P=100)	13.92	13.60	0.61
PSO	17.08	14.07	0.20
GWO	15.48	14.72	0.43
Hybrid	16.19	14.50	0.25

4.5. Eredmények elemzése

Az eredmények elemzésekor erős szezonális hatást figyelhetünk meg: nyáron a magas napelem-termelésnek köszönhetően a napi költség drasztikusan, mintegy 67%-kal csökkenthető ($12.48 \rightarrow 3.93$ RON), míg télen a megtakarítás mértéke alacsonyabb, körülbelül 18% ($42.09 \rightarrow 34.42$ RON). A borús nap a kettő közötti átmenetet képezi. Az algoritmusok teljesítménye is eltérő: a GA ($P=100$) a komplex, változékony esetekben bizonyult a legrobosztusabbnak, míg a GWO télen, a szűkös erőforrások mellett adta a legjobb átlagot. A PSO gyorsasága ellenére nagy keresési térben hajlamos a beragadásra. A hibrid megoldás kiegyensúlyozott teljesítményt nyújtott, bár a többletidő nem minden esetben eredményezett jobb optimumot. A reprodukálhatóság érdekében minden tesztet 20 ismétléssel, determinisztikus seed használatával végeztünk, a futási idők 0.2 és 0.9 másodperc között alakultak.

4.6. Implementációs részletek

A hibrid megoldás PSO és GA fazisokra bontja az optimalizálást. A vezérlő logika egy részlete a 4.1. kódrészletben látható.

4.1. Listing. A hibrid algoritmus vezérlő logikája

```

1 class Hybrid(AlgorithmBase):
    def solve(self):
        # 1. fazis: PSO (felfedezés)
        pso_steps = int(self.max_iter * 0.6)
        pso = PSO(self.func, self.dim, self.lb, self.ub,
6         self.pop_size, pso_steps)
        gb, gb_fit = pso.solve()

        # 2. fazis: GA (finomhangolás)
        ga = GA(self.func, self.dim, self.lb, self.ub,
11         self.pop_size, self.max_iter - pso_steps)
        # ... további lépések

```

4.7. Ábrák

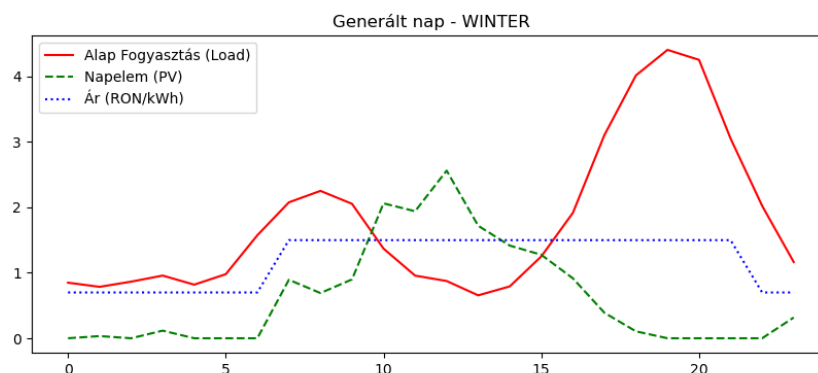
A generált ábrák: `generated_day_*.png`, `results_convergence_*.png`, `results_boxplot_*.png` (winter, summer, cloudy). A konvergencia görbék mutatják a GA lassabb, de stabilabb, és a PSO gyors, de korai stagnálását; a boxplot-ok a 20 futás statisztikai eloszlását ábrázolják.

5. fejezet

Következtetések és Ajánlások

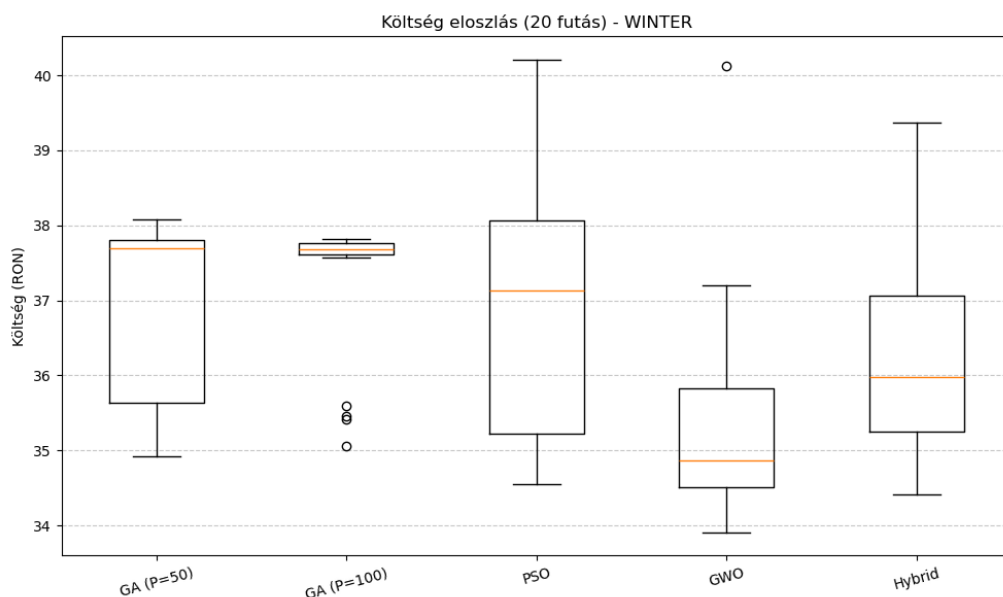
5.1. Egy nap részletes esetelemzése

A 5.1. ábra a téli szcenárió egy tipikus napjának PV-termelés és fogyasztási profiljait mutatja. A 4.1. táblázat eredményei alapján ebben a körülményben a GWO adta a legalacsonyabb átlagos költséget, ami összhangban van a szűkös termelési helyzettel.



5.1. ábra. Téli bemeneti profilok (PV termelés és fogyasztás)

5. FEJEZET: KÖVETKEZTETÉSEK ÉS AJÁNLÁSOK



5.2. ábra. Költség eloszlás a téli scenárióban (20 futás)

5.2. Főbb megállapítások

A kísérletek rávilágítottak a **szezonális különbségek** fontosságára. Nyáron az optimalizáció kulcsa a többlettermelés menedzselése (az akkumulátor töltése vagy visszatáplálás), míg télen a drága csúcsidőszaki fogyasztás minimalizálása a cél. A vizsgált három scenárió (tél, nyár, borús) megfelelően lefedti a valós üzemeltetési körülményeket. Az **algoritmus választást** illetően megállapítható, hogy nincs egyetlen "legjobb" módszer. A GA (P=100) a komplex, változékony esetekben bizonyult robusztusnak, míg a GWO a szigorúbb korlátokkal terhelt téli környezetben működött hatékonyabban. Az **akkumulátor szerepe** különösen a borús tesztelésben mutatkozott meg, ahol kulcsfontosságúnak bizonyult a termelés ingadozásainak kisimításában. A **futási teljesítmény** tekintetében az algoritmusok 0.21 és 0.92 másodperc közötti idő alatt találtak megoldást; a PSO volt a leggyorsabb, de ez gyakran a megoldás minőségének rovására ment.

5.3. Gazdasági megtérülés

5.1. táblázat. Beruházási és megtérülési kalkuláció becslés

Tétel	Költség/Érték
HEMS hardver	≈ 1200 RON
5 kWp napelem rendszer	≈ 15000 RON
10 kWh akkumulátor	≈ 18000 RON
Teljes beruházás	≈ 34200 RON
Éves megtakarítás (becsült)	≈ 6200 RON
Megtérülési idő	≈ 5.5 év

Támogatással (Casa Verde) a megtérülés 3–4 évre csökkenthető.

5.4. Környezeti hatás

A rendszer környezeti hatása jelentős. Az évi 6000 kWh saját napelemes termelés önmagában mintegy 4.8 tonna CO₂ kiváltását eredményezi. Ehhez adódik hozzá a terhelés-eltolásból és a hatékonyságnövelésből származó további 2000 kWh megtakarítás, ami újabb 1.6 tonna kibocsátáscsökkenést jelent. **Összességében így évente nagyságrendileg 6.4 tonna CO₂ megtakarítás érhető el.**

5.5. Jövőbeli kutatási irányok

5.5.1. Energiaközösségek (REC)

Peer-to-peer energia kereskedés, közösségi akkumulátor; potenciális további megtakarítás 10–15%.

5.5.2. Multi-objective optimalizálás

Pareto-front alapú megközelítés: felhasználó választhat költség-komfort görbéről, adaptív súlyozással.

5.5.3. Gépi tanulás integráció

LSTM időjárás előrejelzés, felhasználói szokások tanulása, prediktív karbantartás.

5.5.4. V2G integráció

Elektromos autók mint mobil akkumulátorok: 60 kWh EV kapacitás → 6x nagyobb tárolás, 50–100 RON/hó extra bevétel.

5.6. Implementációs ajánlások

5.6.1. Kis háztartásoknak (< 3 fő)

Kis háztartásoknak egy minimális konfiguráció javasolt, amely egy HEMS vezérlőből és intelligens konnektorokból áll. Ennek beruházási költsége alacsony, 800–1200 RON közé tehető. A rendszerrel elérhető 15–20%-os megtakarítás (évi 500–800 RON) mellett a befektetés várhatóan 1.5–2 év alatt megtérül.

5.6.2. Közepes háztartásoknak (3–5 fő)

Közepes háztartásoknak egy komolyabb rendszer kiépítése ajánlott, amely a HEMS mellett egy 3–5 kWp-es napelemes rendszert és egy 5–10 kWh kapacitású akkumulátort is tartalmaz. Bár a beruházás összege 25000–35000 RON, ez támogatásokkal csökkenthető. A várható megtakarítás ebben az esetben jelentős, 30–50% (évi 3000–5000 RON), így a megtérülési idő támogatással 3–5 évre, támogatás nélkül 6–8 évre becsülhető.

A. függelék

Fontosabb programkódok listája

A.1. HEMS kódrendszer

- `codes/main.py`: szcenáriók futtatása (winter, summer, cloudy), eredmények mentése CSV/JSON-be, ábrák generálása.
- `codes/algorithms.py`: GA, PSO, GWO és hibrid GA-PSO implementációja.
- `codes/hems_problem.py`: SmartHomeEnvironment modell, célfüggvény és korlátkezelés.
- `codes/data_generator.py`: fogyasztási, PV és árprofil generálása szezononként.

A.2. Mintakódrészlet

Az alábbi példa a hibrid algoritmus fő lépéseit illusztrálja:

```
3      # Reinsert elites
      pop = np.vstack((elites, np.clip(offspring, self.lb, self.ub)))
      fitness = np.array([self.func(ind) for ind in pop])

      current_best = np.min(fitness)
      if current_best < best_fit:
          best_fit = current_best
          best_sol = pop[np.argmin(fitness)].copy()
          self.history.append(best_fit)

      return best_sol, best_fit

13 # --- 2. PSO (unchanged) ---class PSO(AlgorithmBase):
      def solve(self):
          w, c1, c2 = 0.6, 1.8, 1.8
          X = np.random.uniform(self.lb, self.ub, (self.pop_size, self.dim))
          V = np.zeros_like(X)
          P_best = X.copy()
```

A. FÜGGELÉK: FONTOSABB PROGRAMKÓDOK LISTÁJA

```

23     P_best_fit = np.array([self.func(x) for x in X])

    g_best = P_best[np.argmin(P_best_fit)].copy()
    g_best_fit = np.min(P_best_fit)
    self.history = [g_best_fit]

    for _ in range(self.max_iter):
        r1, r2 = np.random.rand(self.pop_size, self.dim), np.random.
            rand(self.pop_size, self.dim)
        V = w * V + c1 * r1 * (P_best - X) + c2 * r2 * (g_best - X)
28     X = np.clip(X + V, self.lb, self.ub)

        fits = np.array([self.func(x) for x in X])
        better = fits < P_best_fit
        P_best[better] = X[better]
33     P_best_fit[better] = fits[better]

        current_min = np.min(fits)
        if current_min < g_best_fit:
            g_best_fit = current_min
            g_best = X[np.argmin(fits)].copy()
38     self.history.append(g_best_fit)

    return g_best, g_best_fit

43 # --- 3. GWO (unchanged) ---
class GWO(AlgorithmBase):
    def solve(self):
        wolves = np.random.uniform(self.lb, self.ub, (self.pop_size, self.
            dim))
        fits = np.array([self.func(w) for w in wolves])

48     # Initial sorting
        idx = np.argsort(fits)
        alpha, beta, delta = wolves[idx[:3]].copy()

53     # Elitizmus
        best_sol = alpha.copy()
        best_fit = fits[idx[0]]

```

Irodalomjegyzék

- Fadaee, M. és Radzi, M. Multi-objective optimization of a stand-alone hybrid renewable energy system by using evolutionary algorithms: A review. *Renewable and Sustainable Energy Reviews*, 16(5):3364–3369, 2012. doi: 10.1016/j.rser.2012.02.071.
- Holland, J. H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- Ismail, M., Moghavvemi, M., és Mahlia, T. Genetic algorithm based optimization on modeling and design of hybrid renewable energy systems. *Energy Conversion and Management*, 85: 120–130, 2014. doi: 10.1016/j.enconman.2014.05.047.
- Kennedy, J. és Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN'95*, pages 1942–1948, 1995.
- Kumar, P., Jain, G., és Palwalia, D. K. Genetic algorithm based maximum power tracking in solar power generation. *Conference Paper (UCE, Rajasthan Technical University)*, 2015. Source PDF provided.
- Lazzari, F., Mor, G., Cipriano, J., Solsona, F., Chemisana, D., és Guericke, D. Optimizing planning and operation of renewable energy communities with genetic algorithms. *Applied Energy*, 338:120906, 2023. doi: 10.1016/j.apenergy.2023.120906.
- Mirjalili, S., Mirjalili, S. M., és Lewis, A. Grey wolf optimizer. *Advances in Engineering Software*, 69:46–61, 2014.